

GVR-Coder: A Visual-Feedback Framework for Structured SVG Generation in Complex Document and Meeting Scenarios

Yiming Xu
xym30@mail.ustc.edu.cn
University of Science and Technology
of China
Hefei, Anhui, China

Jihua Kang
kangjihua@bytedance.com
ByteDance Inc.
Shanghai, China

Chunsai Du
duchunsai@bytedance.com
ByteDance Inc.
Shanghai, China

Qifan Zhang
work.qfzhang@gmail.com
ByteDance Inc.
Shanghai, China

Wangqiu Zhou
rafazwq@hfut.edu.cn
Hefei University of Technology
Hefei, Anhui, China

Yiting Wu
wuyitingde@gmail.com
ByteDance Inc.
Shanghai, China

Tianqi Li
tianqi.li@bytedance.com
ByteDance Inc.
Shanghai, China

Qi Song*
qisong09@ustc.edu.cn
University of Science and Technology
of China
Hefei, Anhui, China

Abstract

In demanding professional environments and meeting review scenarios, lengthy text often imposes a high cognitive load. To facilitate efficient information communication, transforming verbose text into logically clear diagrams is essential. Scalable Vector Graphics (SVG) provide an effective representation for this purpose due to their editability and resolution independence. However, current research on Text-to-SVG generation remains hindered by three major challenges: (1) the scarcity of datasets for complex, logic-rich diagrams; (2) the absence of explicit layout priors, which leads to chaotic spatial arrangements; and (3) the lack of fine-grained visual feedback to validate rendered outputs and correct aesthetic defects. To address these challenges, at the data level, we introduce DocMeetSVG-100K, a large-scale SVG dataset tailored for document authoring and meeting review scenarios. At the model level, we propose GVR-Coder, a novel framework designed to generate high-quality logical diagrams from lengthy professional texts. Specifically, we adopt a curriculum-driven rejection sampling fine-tuning to progressively enhance the model’s capability in modeling complex structures, while explicitly incorporating layout constraint knowledge during training. In addition, we introduce reinforcement learning from dual rendering feedback, a mechanism that provides implicit feedback through reward signals to jointly optimize structural complexity and visual aesthetics. Furthermore, we

design a generate-verify-repair agent loop, which improves generation quality through explicit, fine-grained feedback and targeted refinement. Extensive experiments demonstrate that GVR-Coder outperforms competitive baselines and reliably produces logically coherent and visually appealing diagrams. Code and data are available at <https://github.com/CurryNa/GVR-Coder>.

CCS Concepts

• **Computing methodologies** → **Computer vision**.

Keywords

Large Language Models; Scalable Vector Graphics; Generative Models; Reinforcement Learning

ACM Reference Format:

Yiming Xu, Jihua Kang, Chunsai Du, Qifan Zhang, Wangqiu Zhou, Yiting Wu, Tianqi Li, and Qi Song. 2026. GVR-Coder: A Visual-Feedback Framework for Structured SVG Generation in Complex Document and Meeting Scenarios. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26)*, November 10–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3767308.3835261>

1 Introduction

In modern professional office environments, such as document writing [13, 44, 45] and meeting reviews [1, 14, 25], effective information organization and presentation are essential for distilling and communicating complex content. However, lengthy textual documents and dense meeting records often impose considerable cognitive load (Figure 1(a)), making core logical structures and key insights difficult to identify efficiently. In contrast, structured visual representations—such as diagrams, mind maps, and statistical charts—provide a more intuitive means of conveying procedural

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MM '26, Rio de Janeiro, Brazil.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2213-4/2026/11
<https://doi.org/10.1145/3767308.3835261>

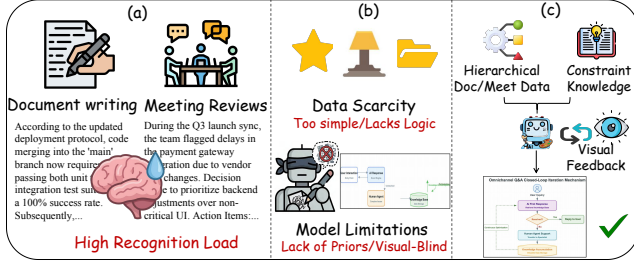


Figure 1: Motivation. (a) Verbose texts impose high cognitive loads. (b) Existing methods struggle with overly simplistic outputs, lack of layout priors, and visual blindness. (c) Our framework resolves this via a curated hierarchical dataset, explicit knowledge, and iterative visual feedback.

logic, task decomposition, and data relationships, thereby improving communication efficiency [38, 42]. Due to its resolution independence and editability, Scalable Vector Graphics (SVG) has become an important technical medium for such visual expressions [3, 15].

While SVGs offer distinct advantages, traditional design workflows [22] rely on professional tools and manual editing, which are labor-intensive and poorly suited for rapid iteration. Therefore, enabling the automatic generation of logically coherent and layout-consistent SVG diagrams from verbose texts remains a critical open research problem. Recently, the proliferation of Artificial Intelligence Generated Content (AIGC) has driven research into automated Text-to-SVG generation, primarily diverging into two paradigms. Optimization-based approaches (e.g., [21, 35, 36, 41]) typically generate SVGs by iteratively optimizing differentiable pixel renderers. However, these methods suffer from low efficiency and often produce disorganized SVG that lack semantic editability. In contrast, recent LLM-based approaches [8, 17, 24, 28, 30, 31, 34, 39] reframe SVG generation as a code synthesis task. By fine-tuning large (visual) language models and introducing specialized tokens to capture SVG structures, these methods significantly boost generation efficiency and code editability, thereby emerging as the mainstream direction in Text-to-SVG research.

Although LLM-based methods have achieved significant progress, they still face three major challenges. **First, the scarcity of high-quality Text-to-SVG datasets specifically curated for complex, high-logic scenarios.** Current research predominantly focuses on simple, isolated icons (Figure 1 (b)), which fails to support the generation of sophisticated diagrams required in scenarios like meeting reviews and document authoring. **Second, the absence of a "designing mind" for constraint-based knowledge guidance.** While current models excel at generating valid code, they inherently lack explicit aesthetic and layout knowledge priors. Consequently, the generated SVGs often suffer from chaotic spatial arrangements. **Finally, the lack of a "perceptive eye" for fine-grained visual feedback.** Existing methods primarily treat SVG generation as a pure code synthesis task but never validate the rendered outputs visually (Figure 1 (b)), which prevents models from perceiving aesthetic defects and performing repairs.

To this end, we propose an innovative solution that augments both the data and model levels (Figure 1(c)). **For Challenge 1**, we

construct DocMeetSVG-100K, a large-scale, finely annotated Text-to-SVG dataset with progressive difficulty levels, filling the gap in document authoring and meeting scenarios. **For Challenge 2**, we introduce a curriculum-driven rejection sampling fine-tuning strategy and incorporate layout knowledge. This design provides "lessons" of varying difficulty to progressively cultivate the model's understanding of SVG diagrams, along with structured priors to provide rigorous layout constraints, thereby averting chaotic arrangements. **For Challenge 3**, we establish a perception system that combines implicit and explicit feedback. During training, we develop a reinforcement learning strategy with dual rendering rewards, which internalizes structural complexity and visual aesthetics into the model's generation intuition via reward signals. During inference, we propose a generate-verify-repair agent loop. The verifier evaluates the rendered SVG to provide fine-grained attribution and suggestions, identifying not just the quality gap but also the underlying causes ('why') and specific errors ('where'). This directs the repair model to perform targeted refinement for high-quality outputs. To sum up, our contributions are:

(1) **DocMeetSVG-100K Dataset:** To alleviate the cognitive load of long-text processing in office scenarios, we construct DocMeetSVG 100K, a large-scale office Text-to-SVG dataset specifically curated for logic diagram design, along with an automated evaluation protocol that combines aesthetic standards and factual consistency.

(2) **GVR-Coder SVG Generation Framework:** We propose GVR-Coder, a novel framework that synergizes progressive knowledge internalizing and iterative visual feedback. By integrating knowledge-enhanced curriculum learning, reinforcement learning with dual-reward signals, and a multi-agent repair loop, GVR-Coder bridges the gap between code synthesis and visual perception.

(3) **Three-Stage Optimization Strategy:** First, we introduce a curriculum-driven training strategy that incorporates layout knowledge to progressively enhance the model's grasp of logic-rich structural priors; second, we design a novel hybrid reward function for reinforcement learning, providing signals for both structural complexity and visual quality optimization; finally, we develop a generate-verify-repair agent loop to iteratively provide fine-grained visual feedback and refine the outputs.

(4) **Experimental Results:** Experimental results demonstrate that GVR-Coder not only outperforms traditional approaches but also surpasses larger-scale LLMs both qualitatively and quantitatively. Further ablation studies provide strong empirical evidence for the effectiveness of the core components. The source code and partial data are provided in the supplementary material.

2 Related Work

2.1 Vector Graphics Generation

SVG generation methods can be broadly divided into optimization-based and LLM-based paradigms. Optimization-based methods use differentiable rasterizers to optimize SVG parameters, such as CLIP-guided sketch [18] generation in CLIPDraw [7] and CLIPasso [23], or diffusion-driven iterative refinement in VectorFusion [10] and SVGDreamer [37]. However, these methods often produce entangled paths with limited semantic editability, while their iterative optimization also reduces efficiency. Recently, LLM-based methods [6, 9, 12, 16, 19, 20, 27, 40] have become increasingly attractive

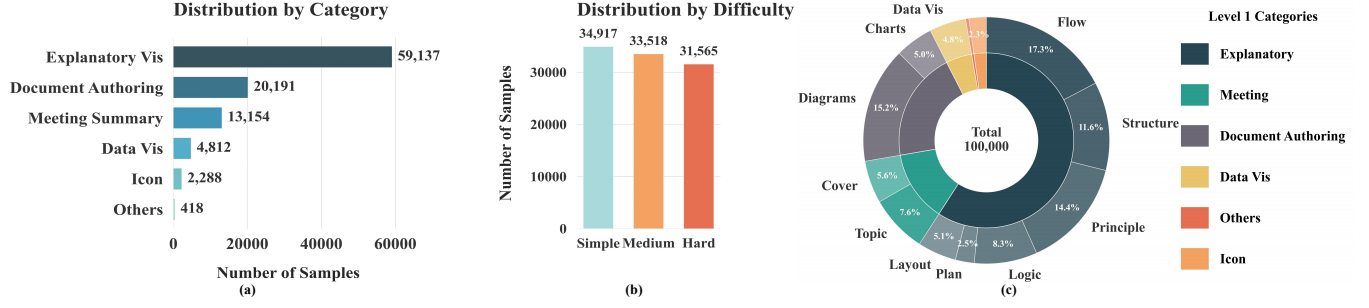


Figure 2: Statistics of our dataset DocMeetSVG-100K. (a) shows the sample count across six primary categories. (b) presents the volume of samples categorized by difficulty levels. (c) illustrates the proportional breakdown of sub-categories.

due to their semantic understanding and code generation ability. LLM4SVG [33] and OmniSVG [39] improve controllability through semantic tokens or structural-geometric decoupling, while ReasonSVG [32] and SVGGen [24] further introduce reasoning or reward-based optimization. Despite these advances, existing approaches remain limited, struggling either with outputs that lack structural diversity and complex logical organization, or with a fundamental absence of SVG layout priors and visual feedback.

2.2 SVG Datasets

While early works such as FIGR-8-SVG [5] and SVG-Icons8 [2] provide massive collections of basic icons, they are typically restricted to monochrome, single-object designs lacking compositional structure. To enhance visual expressiveness, datasets like ColorSVG-100K [4], LLM4SVG [34], and MMSVG-2M [39] introduced colorful vector graphics and intricate anime designs, leveraging multimodal models to construct text prompts. Recently, SVG-Stack [19], SVG-Sophia [26], and SVG-1M [24] have further scaled text-SVG alignment to the million level, incorporating semantic annotations and Chain-of-Thought (CoT) enhancement. However, while these datasets [11, 29] have advanced the field in terms of scale and artistic visual richness, they are fundamentally oriented toward general UI assets. They lack topological relationships (e.g., flowcharts), and strict hierarchical nesting (e.g., mind maps) required in professional office settings.

3 DocMeetSVG-100K Dataset Construction

To bridge the structural and logical gaps in existing SVG datasets, we introduce DocMeetSVG-100K, a large-scale dataset specifically designed for professional document and meeting scenarios. To ensure authenticity and diversity, we collect captions from real-world sources, including meeting records, professional office documents, and the Arena conversation dataset. Each input is formulated as a structured tuple of raw text, application scenario, and target diagram type. Since different diagram categories require different structural and layout patterns, we adopt type-specific construction pipelines rather than a single unified prompt template. For each caption, Gemini-3-Pro generates two independent SVG candidates, which are evaluated by a Judge Model from visual, layout, topology, and semantic-consistency perspectives. Imperfect samples are further refined by a specialized Repair Model until they meet the

predefined quality standards. After generation and repair, we remove duplicates and conduct stratified human verification, where 30% of samples in each difficulty level are manually audited for visual quality, semantic faithfulness, and layout correctness. For evaluation, the 320 test cases are collected from real documents and meeting records outside the training set, and all test samples are fully manually validated before benchmarking. As shown in Figure 2, DocMeetSVG-100K contains 100,000 Text-SVG pairs across six primary categories with balanced difficulty levels, supporting logic-rich SVG diagram generation.

4 Method

In modern professional scenarios, such as document authoring and post-meeting reviews, users are often overwhelmed by lengthy and unstructured texts. The goal of our task is to automatically transform these verbose inputs into logically clear and visually intuitive SVG diagrams, thereby organizing procedural logic, and explicitly presenting core themes. To achieve this, we propose GVR-Coder, a visual-feedback framework tailored for complex SVG generation. As illustrated in Figure 3, our framework integrates three core stages to achieve logically rigorous and aesthetically superior outputs.

4.1 Curriculum-driven Rejection Sampling Fine-tuning with Knowledge Augmentation

As described in Sec. 3, our basic idea is to harness the powerful generation capabilities of LLMs to assist in constructing a high-quality training dataset. However, while advanced models like Gemini-3-Pro demonstrate remarkable potential in code synthesis, their success in generating logically rigorous SVGs is often stochastic.

Therefore, to transform this generative diversity into high-quality supervision signals, we decide to employ rejection sampling to improve the quality of the fine-tuning data. Specifically, for each input text caption, we first prompt the model to generate a set of candidate responses. Then, an automated verifier filters these candidates, rejecting those with defects and retaining only the perfect outputs to construct a high-confidence Supervised Fine-Tuning (SFT) dataset. Moreover, directly training the model on a uniformly mixed dataset can easily lead to convergence failure. To this end, we introduce a curriculum learning strategy to progressively enhance the model’s learning capability, preventing it from being overwhelmed by overly difficult data at the initial stage.

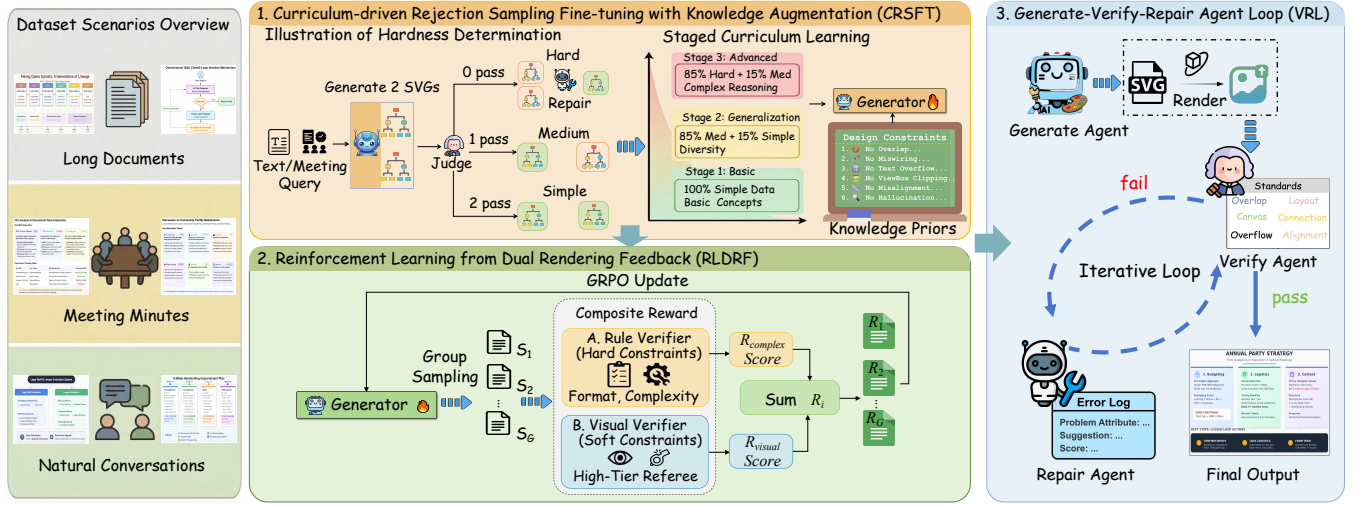


Figure 3: The overall framework: Dataset scenarios and GVR-Coder workflow.

Combining these insights, we propose Curriculum-driven Rejection Sampling Fine-Tuning (CRSFT). In our implementation, we deliberately set the number of responses to exactly two to balance generative diversity with computational overhead. Conveniently, the rejection sampling process naturally stratifies the data into three difficulty levels based on the sampling success ratio: (1) **Simple**. Both samples are perfect, covering fundamental SVG structures and attributes. (2) **Medium**. Only one sample is perfect, introducing generative diversity. (3) **Hard**. Both samples exhibit flaws, typically corresponding to complex tasks or multi-step reasoning. These instances are optimized using a specialized *Repair Model*.

Following the cognitive principle of “learning from simple to complex”, we implement a three-stage curriculum learning. Stage 1 (Basic) utilizes 100% simple data to build a foundational understanding of basic SVG syntax and design. Stage 2 (Generalization) introduces a mix of 85% medium and 15% simple data to enhance generative diversity and cross-scenario generalization. Finally, Stage 3 (Advanced) incorporates 85% hard and 15% medium data to strengthen the model’s generative capability for complex logical tasks.

While current models excel at generating valid code, their lack of layout priors often leads to chaotic arrangements. To compensate for this inherent deficiency, we introduce a constraint-aware knowledge augmentation module. Explicit design constraints—including priors such as collision avoidance (via bounding box constraints), connectivity logic (via anchor-point alignment), and factual fidelity (via anti-hallucination rules)—are injected into the prompt to guide the model’s spatial reasoning and minimize structural defects.

4.2 Reinforcement Learning from Dual Rendering Feedback

While CRSFT improves basic generation ability, token-level likelihood optimization alone cannot ensure visual fidelity. To address this, we propose Reinforcement Learning from Dual Rendering Feedback (RLDRF), which employs GRPO to optimize a hybrid reward balancing aesthetics and structural complexity. The aesthetic

reward improves visual quality but may encourage overly simplified outputs to avoid layout errors. Therefore, we introduce a complexity reward to prevent such degeneration by maintaining element density comparable to the ground truth. This balance enables the model to generate diagrams that are both visually appealing and structurally rich. The reward components are defined as follows.

(1) *Aesthetic Reward*: For each generated SVG, we first perform a validity check. Non-renderable code is penalized with a score of 0. Valid images are evaluated by a high-tier VLM (Gemini-3-pro) across six dimensions: element overlap, connection issues, text overflow, layout clutter, alignment styles, and content occlusion. The reward is calculated using a deductive scoring system:

$$R_{\text{visual}} = \frac{\max(0, 6 - N_{\text{errors}})}{6}, \quad (1)$$

where N_{errors} represents the number of detected visual defects.

(2) *Complexity Reward*: We introduce a complexity matching reward, which counts structural tags n (including *path*, *circle*, *line*, etc.) in both generated and reference code. A threshold of 0.8 is applied to ensure the generated result matches the target complexity:

$$R_{\text{complex}} = \begin{cases} 1.0 & \text{if } n_{\text{gen}} \geq 0.8 \cdot n_{\text{ref}} \\ \frac{n_{\text{gen}}}{0.8 \cdot n_{\text{ref}}} & \text{otherwise} \end{cases}. \quad (2)$$

This approach ensures the generated SVG maintains a density of elements comparable to the ground truth.

The total reward R combines the aforementioned rewards:

$$R_{\text{total}} = R_{\text{visual}} + R_{\text{complex}}. \quad (3)$$

Through GRPO optimization on 8,000 hard samples, the model is capable of generating diagrams that meet aesthetic expectations while maintaining structural richness.

4.3 Generate-Verify-Repair Agent Loop

While the RLDRF stage effectively aligns the model’s global generation policy with visual preferences, it fundamentally relies on

scalar reward signals. Consequently, the model learns whether a generated SVG possesses high overall quality, but lacks explicit attribution regarding where or why specific local defects occur, leaving it with no opportunity to repair its single-pass outputs. To address this limitation during inference, we introduce a multi-agent loop. This framework simulates a pedagogical process of “student submission $\rightarrow$ teacher feedback $\rightarrow$ student revision $\rightarrow$ teacher score”. Unlike the implicit score-based guidance in training, this iterative framework utilizes a verifier to provide fine-grained, attribute-level visual feedback, offering actionable suggestions that guide a specialized repair agent to perform targeted refinements. The core components and iterative logic are as follows:

Generate Agent: Powered by the model optimized in the preceding CRSFT and RLDRF stages, this agent serves as the starting point to receive a constraint-refined instruction (Caption) and synthesize an initial SVG code $P_0 = \text{Agent}_{\text{gen}}(\text{Caption})$. To bridge code and visual perception, the generated program is subsequently rendered into a visual image $I_t = \text{Agent}_{\text{render}}(P_t)$ at each t -th iteration.

Verify Agent: Leveraging the multimodal capabilities of Gemini-3-Pro, this agent analyzes the original caption, the SVG code P_t , and the rendered image I_t , in order to detect logical deviations or visual artifacts. The evaluation process is formulated as:

$$\{D_t, F_t, S_t\} = \text{VLM}_{\text{verify}}(\text{Caption}, P_t, I_t), \quad (4)$$

where D_t denotes the decision result (pass/fail), F_t represents feedback information containing problem attributes and revision suggestions, and S_t denotes the current evaluation score.

Repair Agent: To equip the model with robust code revision capabilities, this agent is trained on a curated dataset of approximately 20,000 repair instances. This training corpus is constructed from two complementary sources: automated corrections generated by Gemini-3-Pro and manual refinements executed by human annotators, who adjust visual components using the Inkscape software. During the loop, the agent takes the current SVG P_t and the verifier’s feedback F_t as inputs to reconstruct and optimize the SVG accordingly. The process is defined as:

$$P_{t+1} = \text{Agent}_{\text{repair}}(P_t, F_t). \quad (5)$$

The loop is executed at most three times, as empirical observations indicate that performance gains plateau beyond this point, with additional iterations offering negligible incremental utility.

5 Experiment

To evaluate the performance of our proposed method for SVG generation tasks, we conduct comprehensive experiments on the constructed DocMeetSVG-100K dataset. Specifically, we reserve 320 samples as an independent test set, while utilizing the remaining data for training. To systematically assess model performance across diverse real-world scenarios, this test set is composed of four distinct benchmarks: **Arena_human_50**, an open-ended multilingual dialogue-driven setting; **Meeting_cover_100**, focusing on structured meeting cover layout generation; **Meeting_topic_50**, targeting abstract topic-to-structure modeling; and **Svg_with_rules_120**, which imposes strict topological and layout constraints derived from professional documentation and academic papers, representing a highly creative generation scenario.

We provide the complete implementation in our code repository, where the training details (including full-parameter fine-tuning, GRPO training, and the ms-swift [43] framework) are specified.

5.1 Evaluation Protocols

5.1.1 Automated Evaluation System. (1) Aesthetic and Normative Scoring: Standard Text-to-Image metrics such as FID and CLIPScore often correlate poorly with the logical and topological requirements of document-level diagrams. We therefore adopt Gemini-3-Pro as VLM-as-Judge to provide nuanced visual feedback.

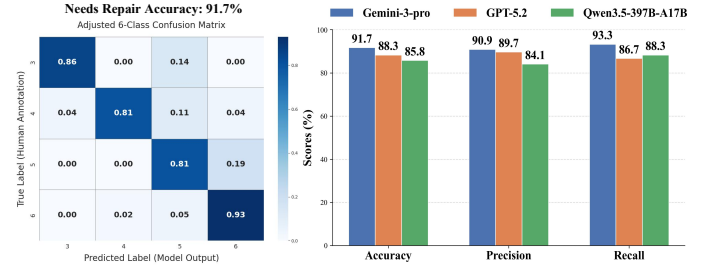


Figure 4: Verifier’s Reliability (Left) and Robustness (Right).

To bridge the gap between automated checking and manual review, we classify visual defects into three dimensions: boundaries and completeness, topology and logic, and layout and aesthetics. These dimensions cover six error types: canvas clipping, content occlusion, text overflow, connector issues, misalignment, and cluttered layout. We then adopt a deduction-based scoring mechanism:

$$\text{Score} = \max(0, 6 - N), \quad (6)$$

where N represents the number of detected errors.

(2) **Factuality Scoring:** The score is assigned to measure semantic alignment across three core dimensions: Zero Hallucination, No Omission, and Logical Consistency. 1 Point: The SVG satisfies all three criteria, faithfully reflecting the caption without redundant content. 0 Point: The image fails in any dimension. For more detailed prompts, please refer to the supplementary materials.

5.1.2 Reliability of VLM-as-a-Judge. We compare the VLM’s performance against human experts on 200 samples (Figure 4). The VLM-as-a-judge achieved a 90% agreement rate with humans in aesthetic and normative “repair vs. no repair” decisions. Fine-grained scoring showed strong alignment, with diagonal proportions in the six-class confusion matrix ranging from 0.81 to 0.93. Regarding factual consistency, it achieved an F1 score of 0.87 compared to human annotations. Meanwhile (Right), various models show consistently high performance, confirming our protocol is model-agnostic and supports cost-effective open-source deployment.

5.2 Quantitative Analysis

As shown in Table 1 and Figure 6, GVR-Coder variants consistently outperform all baselines. Traditional methods fail in document-level tasks ($\text{Ar} \approx 0$), as their icon-centric training data lacks the priors for complex hierarchical organization.

Compared to SOTA closed-source models, GVR-Coder achieves superior accept ratio in open-ended, multilingual dialogues (82.0%

Table 1: Quantitative comparison with SOTA models. The **gray** and **blue** color rows denote the performances of open-source models and GVR-Coder. Best in bold, runner-up underlined. In addition to the *Average score* (As), we emphasize the perfect *Acceptance rate* (Ar), which measures the proportion of fully usable outputs. And scores are categorized by quality: 0-2 (Low), 3-4 (Medium), and 5-6 (High). Furthermore, we introduce *SVG Validity* (first-pass syntactic validity) to assess code-level robustness.

Model	Arena_human_50					Meeting_cover_100					Meeting_topic_50					Svg_with_rules_120					SVG
	As	Ar	Low	Med.	High	As	Ar	Low	Med.	High	As	Ar	Low	Med.	High	As	Ar	Low	Med.	High	Valid.(%)
Traditional SVG Models																					
SVGGen	80.0	16.0	0	17	33	72.0	0.0	0	68	32	78.0	0.0	0	15	35	76.0	0.0	0	52	68	89.9
LLM4SVG	60.0	2.0	5	35	8	50.0	0.0	23	75	2	56.0	0.0	6	41	3	50.0	0.0	30	85	5	76.5
Open-Source Models																					
InternVL-3.5-38B	70.0	4.0	1	31	18	61.0	2.0	3	91	6	66.0	0.0	0	43	7	64.0	3.3	1	98	21	88.7
Deepseek-V3.2	74.7	24.0	0	30	20	67.0	4.0	2	74	24	68.0	8.0	0	41	9	66.5	6.0	3	90	27	92.6
Qwen3.5-397B-A17B	80.3	30.0	0	19	31	81.7	29.0	1	30	69	84.0	44.0	0	19	31	81.3	35.0	2	42	76	92.8
Closed-Source Models																					
GPT-5.1	70.3	4.0	0	33	17	68.8	4.0	0	74	26	71.0	8.0	0	34	16	68.1	5.8	2	87	31	95.6
Gemini-2.5-pro	78.0	18.0	0	23	27	84.3	44.0	0	35	65	71.3	12.0	0	36	14	78.9	28.0	0	56	64	94.4
GLM-5	79.0	28.0	0	25	25	84.3	38.0	0	30	70	83.7	34.0	0	16	34	83.2	34.2	0	38	82	93.0
Kimi-k2.5	81.7	28.0	0	17	33	87.5	46.0	0	20	80	86.0	48.0	0	13	37	81.4	29.2	0	45	75	95.9
GPT-5.5	78.3	36.0	0	27	23	86.3	55.0	0	34	66	80.7	42.0	0	25	25	72.9	27.5	2	77	41	94.8
Gemini-3-pro	83.3	38.0	0	17	33	92.8	69.0	0	11	89	89.0	56.0	0	11	39	86.0	50.0	0	34	86	97.2
GVR-Coder																					
Qwen3-14B	66.0	6.0	1	41	8	67.5	1.0	0	74	26	64.7	2.0	0	42	8	62.1	1.0	3	101	16	76.3
GVR-Coder-14B-RSFT	87.3	42.0	0	11	39	94.0	71.0	0	9	91	89.0	60.0	0	12	38	87.4	52.5	0	32	88	96.9
GVR-Coder-14B-RLDRF	93.3	74.0	0	5	45	95.0	74.0	0	6	94	96.5	82.0	0	3	47	88.0	56.6	1	32	87	99.4
GVR-Coder-14B-VRL	96.0	82.0	0	1	49	97.7	90.0	0	4	96	97.0	88.0	0	3	47	95.3	81.6	0	12	108	N/A
Qwen3-32B	70.0	6.0	1	37	12	73.8	13.0	0	61	39	70.7	10.0	0	34	16	66.0	3.3	4	92	24	89.0
GVR-Coder-32B-RSFT	89.0	48.0	0	7	43	95.0	68.0	0	6	94	93.0	70.0	0	7	43	86.0	48.3	0	36	84	97.8
GVR-Coder-32B-RLDRF	94.0	76.0	0	4	46	94.8	75.0	0	13	87	97.0	84.0	0	2	48	88.0	54.2	0	29	91	99.7
GVR-Coder-32B-VRL	94.7	82.0	0	7	43	100.0	100.0	0	0	100	97.3	90.0	0	3	47	96.9	88.3	0	7	113	N/A

on Arena), strict constraint tasks (88.0% on Svg_with_rules), and meeting scenarios (up to 100%). These results highlight our model’s reliability in topic extraction and hierarchical organization, while effectively mitigating text overflow, element overlap, and connection conflicts. Regarding factual consistency, GVR-Coder ranks top in meeting datasets and second in open-ended settings, where semantic boundaries are less explicit (Figure 7).

Performance gains follow a clear stage-wise pattern: (1) RSFT establishes a robust foundation (As > 86.0) through curriculum learning; (2) RLDRF bridges the modality gap using visual rewards, significantly boosting Ar; and (3) VRL resolves specific layout defects via fine-grained visual feedback and targeted repair. Furthermore, GVR-Coder achieves a first-pass rendering success rate > 99% by enforcing syntactic constraints during RL, substantially reducing regeneration costs in real-world deployment.

5.3 Cross-Evaluator Consistency and Quality–Latency Trade-off

Although Gemini-3-Pro is used as the primary VLM-as-Judge, our results are not evaluator-specific. As shown in Table 2, re-evaluation with Gemini-3-Pro, GPT-5.2, and Qwen3.5-397B yields stable rankings, with GVR-Coder consistently achieving the highest acceptance rate. This suggests that the improvement is not an artifact of a single Gemini-based scoring protocol.

As shown in Fig. 5, GVR-Coder achieves 96% acceptance with only 7.02s amortized latency, which is lower than Gemini-3-Pro

Table 2: Cross-Verifier robustness under different VLMs.

Model	Gemini-3-Pro		GPT-5.2		Qwen3.5-397B	
	As	Ar	As	Ar	As	Ar
GLM-5.2	83.0	34%	81.8	38%	82.0	35%
GPT-5.1	69.1	5%	68.7	5%	65.2	4%
Gemini-3-Pro	88.5	55%	88.1	56%	87.4	53%
GVR-Coder-14B-RSFT	89.0	57%	90.0	58%	89.0	56%
GVR-Coder-14B-RLDRF	<u>91.1</u>	<u>65%</u>	<u>90.7</u>	<u>64%</u>	<u>90.0</u>	<u>62%</u>
GVR-Coder-14B-VRL	98.1	96%	99.1	97%	97.6	94%

single-pass generation (8.47s) and close to GPT-5.5 (6.61s). The Pareto frontier further shows that the first three repair rounds steadily improve acceptance with limited latency overhead, while the fourth round brings almost no additional gain. This supports our choice of limiting VRL to three iterations and demonstrates a favorable quality–latency trade-off.

Table 3: Ablation study of the GVR-Coder framework.

Model	Average Score ↑	Accept Ratio ↑	Usability Rate ↑
Base Model (Vanilla SFT)	86.2	49.1	76.0
+ Curriculum Learning	88.5	54.6	78.1
+ Knowledge Injection	90.2	56.8	80.3
+ RLDRF	91.1	67.2	83.0
+ VRL (Ours)	97.6	95.8	97.2

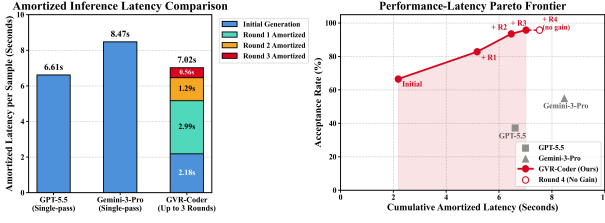


Figure 5: Inference efficiency and performance trade-off.

5.4 Ablation study

We start from the base model and incrementally add each component to demonstrate their cumulative contributions (Table 3). We examine these improvements through the lens of two core metrics: Accept Ratio (Score 6) and Usability Rate (Score ≥ 5).

Analysis of Curriculum Learning. We graded instruction data difficulty based on rejection sampling success rates. With this "easy-to-hard" strategy, the accept ratio increases from the 49.0% baseline to 55.0%. It allows the model to steadily build capabilities from simple primitives to complex topologies. This prevents the model from being overwhelmed by too hard tasks in early stages, significantly enhancing convergence stability and alignment quality.

Analysis of Constraint Knowledge Injection. To mitigate text models' spatial perception deficiencies, we injected constraint knowledge (e.g., collision avoidance and overflow prevention) during training. Incorporating this prior further boosts the accept ratio from 54.6% to 56.8%. This confirms that purely data-driven methods struggle with strict geometric constraints. Converting defects into design rules provides clear decision boundaries.

Analysis of RLDRF. To enhance complex diagram generation, we apply GRPO on 8,000 hard samples. Ablation results show that RLDRF improves the accept ratio from 56.8% to 67.2%, highlighting the necessity of implicit visual feedback via reward signals when handling complex scenarios. A more detailed analysis of the reward design and its effects is provided in the Supplementary Material.

Analysis of Generate-Verify-Repair Agent Loop. As shown in the Sankey diagram (Figure 8), we track score transitions from initial generation to three repair rounds. The accept ratio increases from 66.6% to 82.8%, 93.4%, and 95.9% across iterations. Although gains gradually saturate and a few samples degrade due to code disruption, the overall trend confirms the effectiveness of fine-grained visual feedback and targeted repair.

5.5 Qualitative Analysis

5.5.1 Case Study. As shown in Figure 9(a), our iterative agent loop rectifies subtle defects like element overlap, achieving pixel-level precision that single-pass models lack. Moreover, qualitative comparisons (Figure 9(b)) show our method outperforms closed-source LLMs by avoiding alignment failures and demonstrating superior aesthetic appeal. Notably, via the complexity reward, it prevents over-simplification to ensure robust logic visualization.

5.5.2 Human Evaluation and User Study. To evaluate real-world utility, 10 UI designers scored generated SVGs from 1 to 10 in terms of semantic match and visual quality. Each image was independently evaluated by two designers, with a senior designer

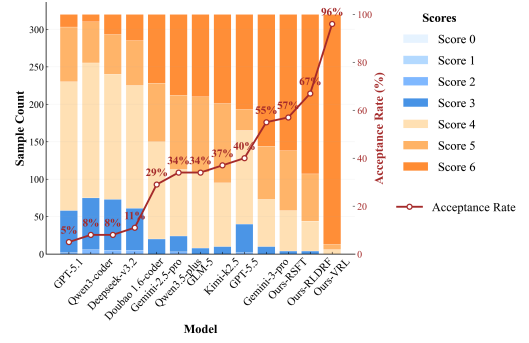


Figure 6: Score distribution and Acceptance Rate on 320 evaluation samples.

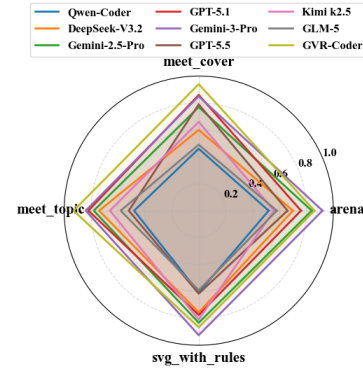


Figure 7: Factuality Performance Radar Map.

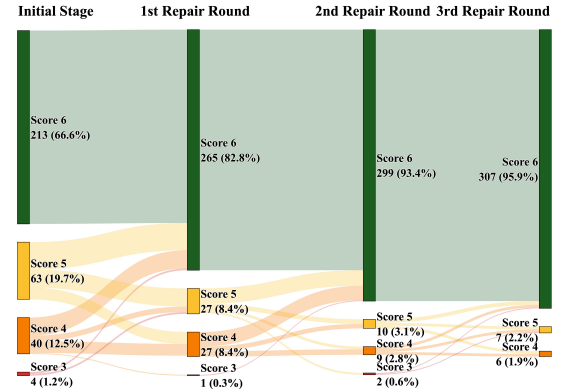
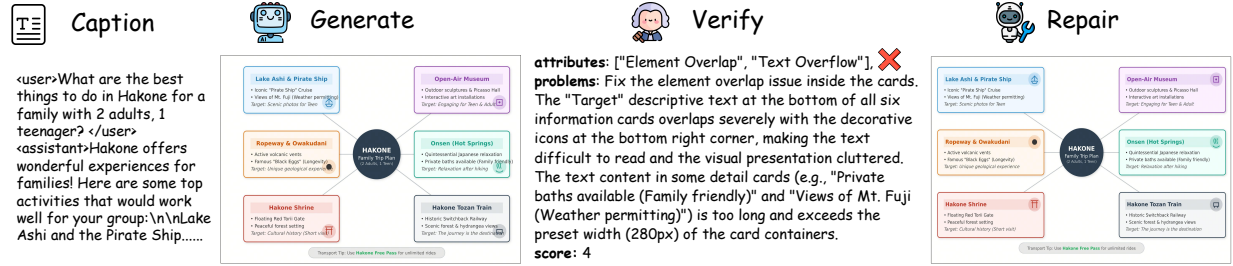


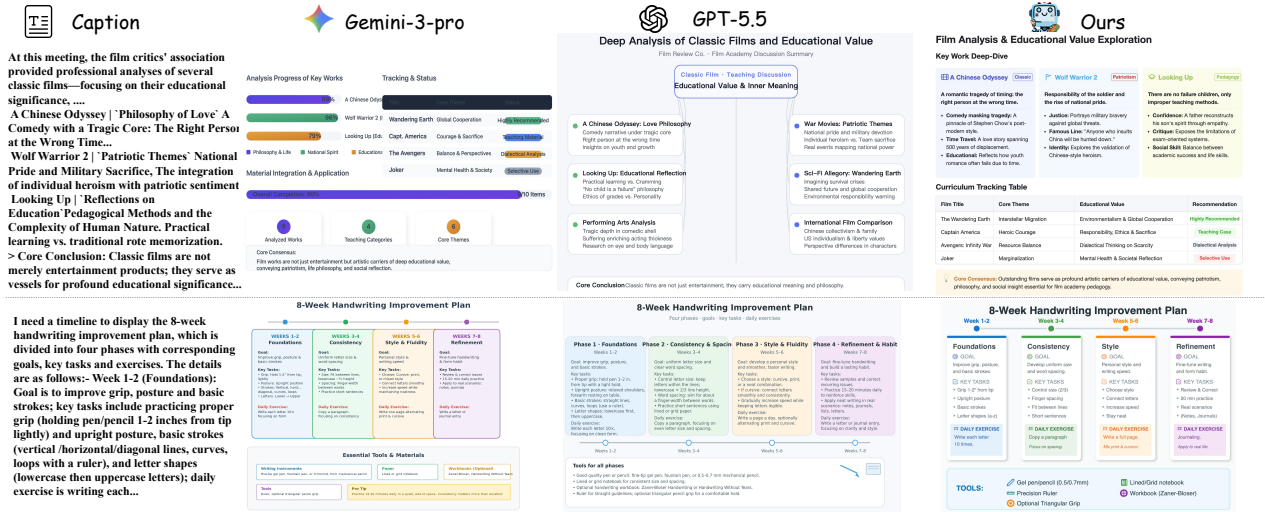
Figure 8: Score transitions across iterative repair rounds.

Table 4: Human Evaluation. Samples are scored up to a maximum of ten based on criteria detailed in the Supplementary Material. The final reported values are the average scores.

Metric/Method	Deepseek-V3.2	GPT-5.1	GPT-5.5	Gemini-3-pro	GVR-Coder
Semantic Match	7.77	8.78	8.26	8.89	9.05
Visual Quality	6.71	7.95	8.34	8.61	8.82



(a) Qualitative visualization of the Generate-Verify-Repair loop.



(b) Qualitative comparison with SOTA methods.

Figure 9: Qualitative performance of GVR-Coder. (Due to space constraints, please refer to the Supplementary Material for high-resolution versions with legible details).

resolving discrepancies. As shown in Table 4, GVR-Coder achieves the highest average scores in both Semantic Match (9.05) and Visual Quality (8.82), indicating that our method better balances factual consistency and visual aesthetics. Conversely, existing LLMs show a 'seesaw effect': Gemini-3 prioritizes semantic logic over aesthetic layouts, while GPT-5.5 tends to sacrifice semantic accuracy for better visual appeal.

Beyond expert scoring, we further conduct a blind user study to examine whether the generated diagrams reduce cognitive load in practice. Specifically, 20 participants with diverse ages rank diagrams generated by GVR-Coder, Gemini-3-Pro, GLM-5, and GPT-5.1 on 50 high-cognitive-load texts according to clarity and difficulty reduction. GVR-Coder obtains the highest average ranking score (3.8 vs. 2.8/2.0/1.6), suggesting that its improvements translate into practical readability gains beyond visual quality.

6 Conclusions

Generating high-quality, editable diagrams from long-form meeting minutes and document texts holds great promise for office scenarios, while simultaneously posing new challenges to existing SVG generation techniques. To advance this line of research, we

introduce DocMeetSVG, a benchmark comprising 100,000 high-quality "office text-svg" pairs that cover diverse document genres and meeting styles. Building upon this, we further propose GVR-Coder, an SVG agent framework grounded in visual-feedback. We first design a knowledge-enhanced curriculum learning strategy, where structural constraints and hierarchical data jointly guide the model to progressively acquire stable generation capability. We then introduce dual rendering reward signals and integrate them with GRPO-based reinforcement learning to improve completeness and visual appeal. Finally, we develop an agent loop that leverages fine-grained visual feedback and targeted suggestions to iteratively refine outputs for practical deployment. Through both automatic evaluation and human assessment, we demonstrate that GVR-Coder produces diagrams that meet office standards, achieving both aesthetic layout quality and semantic fidelity.

Acknowledgments

The research was partially supported by the China National Natural Science Foundation with no. 92567301 and 62132018, and Anhui Provincial Science and Technology Innovation Program 202523009050019 and 202523009050015.

References

- [1] Sumit Asthana, Sagi Hilleli, Pengcheng He, and Aaron Halfaker. 2025. Summaries, Highlights, and Action items: Design, implementation and evaluation of an LLM-powered meeting recap system. *Proceedings of the ACM on Human-Computer Interaction* 9, 2 (2025), 1–29.
- [2] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. 2020. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems* 33 (2020), 16351–16361.
- [3] Siqi Chen, Xinyu Dong, Haolei Xu, Xingyu Wu, Fei Tang, Hang Zhang, Yuchen Yan, Linjuan Wu, Wenqi Zhang, Guiyang Hou, et al. 2025. Svgenius: Benchmarking llms in svg understanding, editing and generation. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 13289–13296.
- [4] Zehao Chen and Rong Pan. 2024. SVGBuilder: Component-Based Colored SVG Generation with Text-Guided Autoregressive Transformers. *arXiv preprint arXiv:2412.10488* (2024).
- [5] Louis Clouâtre and Marc Demers. 2019. Figr: Few-shot image generation with reptile. *arXiv preprint arXiv:1901.02199* (2019).
- [6] Google DeepMind. 2025. Gemini 3.0. <https://deepmind.google/technologies/gemini/>.
- [7] Kevin Frans, Lisa Soros, and Olaf Witkowski. 2022. Clipdraw: Exploring text-to-drawing synthesis through language-image encoders. *Advances in Neural Information Processing Systems* 35 (2022), 5207–5218.
- [8] Adam Hazimeh, Ke Wang, Mark Collier, Gilles Baechler, Efi Kokipoulou, and Pascal Frossard. 2026. Semantic Document Derendering: SVG Reconstruction via Vision-Language Modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 4636–4644.
- [9] Qijia He, Xunmei Liu, Hammaad Memon, Ziang Li, Zixian Ma, Jaemin Cho, Jason Ren, Daniel S Weld, and Ranjay Krishna. 2026. VFIG: Vectorizing Complex Figures in SVG with Vision-Language Models. *arXiv preprint arXiv:2603.24575* (2026).
- [10] Ajay Jain, Amber Xie, and Pieter Abbeel. 2023. Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1911–1920.
- [11] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. 2022. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533* (2022).
- [12] Maikel Leon. 2025. GPT-5 and open-weight large language models: Advances in reasoning, transparency, and control. *Information Systems* (2025), 102620.
- [13] Hao Li, Yuhao Wang, Wenning Hao, Pingping Zhang, Dong Wang, and Huchuan Lu. 2026. RAGTrack: Language-aware RGBT Tracking with Retrieval-Augmented Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 28179–28189.
- [14] Hao Li, Yuhao Wang, Xiantao Hu, Wenning Hao, Pingping Zhang, Dong Wang, and Huchuan Lu. 2026. Cadtrack: Learning contextual aggregation with deformable alignment for robust rgbt tracking. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 6109–6117.
- [15] Jinke Li, Jiarui Yu, Chenxing Wei, Hande Dong, Qiang Lin, Liangjing Yang, Zhicai Wang, and Yanbin Hao. 2025. Unisvg: A unified dataset for vector graphic understanding and generation with multimodal large language models. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 13156–13163.
- [16] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [17] Chaohu Liu, Kun Yin, Haoyu Cao, Xinghua Jiang, Xin Li, Yinsong Liu, Deqiang Jiang, Xing Sun, and Linli Xu. 2024. Hrvda: High-resolution visual document assistant. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 15534–15545.
- [18] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [19] Juan A Rodriguez, Shubham Agarwal, Issam H Laradji, Pau Rodriguez, David Vazquez, Christopher Pal, and Marco Pedersoli. 2023. Starvector: Generating scalable vector graphics code from images. *arXiv preprint arXiv:2312.11556* (2023).
- [20] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. 2026. Kimi K2. 5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* (2026).
- [21] Vikas Thamiharasan, Difan Liu, Matthew Fisher, Nanxuan Zhao, Evangelos Kalogerakis, and Michal Lukac. 2024. Nivel: Neural implicit vector layers for text-to-vector generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4589–4597.
- [22] Yilin Tian, Yue Shang, Chaoxu Kang, and Kaibo Zhao. 2025. A Review of Text-Guided Vector Graphics Generation. In *2025 21st International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. IEEE, 1–6.
- [23] Yael Vinker, Ehsan Pajouheshgar, Jessica Y Bo, Roman Christian Bachmann, Amit Haim Bermano, Daniel Cohen-Or, Amir Zamir, and Ariel Shamir. 2022. Clipasso: Semantically-aware object sketching. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–11.
- [24] Feiyu Wang, Zhiyuan Zhao, Yuandong Liu, Da Zhang, Junyu Gao, Hao Sun, and Xuelong Li. 2025. Svgen: Interpretable vector graphics generation with large language models. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 9608–9617.
- [25] Gaikang Wang, Qiwen Zhao, Zhongwen Zhou, and Yibang Liu. 2025. Research on real-time multilingual transcription and minutes generation for video conferences based on large language models. *Spectrum of Research* 5, 1 (2025).
- [26] Haomin Wang, Qi Wei, Qianli Ma, Shengyuan Ding, Jinhui Yin, Kai Chen, and Hongjie Zhang. 2026. Reliable Reasoning in SVG-LLMs via Multi-Task Multi-Reward Reinforcement Learning. *arXiv preprint arXiv:2603.16189* (2026).
- [27] Haomin Wang, Jinhui Yin, Qi Wei, Wenguang Zeng, Lixin Gu, Shenglong Ye, Zhangwei Gao, Yaohui Wang, Yanting Zhang, Yuanqi Li, et al. 2025. Internsvg: Towards unified svg tasks with multimodal large language models. *arXiv preprint arXiv:2510.11341* (2025).
- [28] Junhui Wang, Gongjie Zhang, Qunhao Qian, Junlong Gao, Deli Zhao, and Ran Xu. 2025. Robosvg: A unified framework for interactive svg generation with multi-modal guidance. *arXiv preprint arXiv:2510.22684* (2025).
- [29] Yuqing Wang, Yizhi Wang, Longhui Yu, Yuesheng Zhu, and Zhouhui Lian. 2023. Deepvecfont-v2: Exploiting transformers to synthesize vector fonts with higher quality. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 18320–18328.
- [30] Ronghuan Wu, Wanchao Su, and Jing Liao. 2025. Chat2svg: Vector graphics generation with large language models and image diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 23690–23700.
- [31] Ronghuan Wu, Wanchao Su, Kede Ma, and Jing Liao. 2023. Iconshop: Text-guided vector icon synthesis with autoregressive transformers. *ACM Transactions on Graphics (TOG)* 42, 6 (2023), 1–14.
- [32] Ximing Xing, Yandong Guan, Jing Zhang, Dong Xu, and Qian Yu. 2025. Reason-SVG: Hybrid Reward RL for Aha-Moments in Vector Graphics Generation. *arXiv preprint arXiv:2505.24499* (2025).
- [33] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, and Qian Yu. 2024. Empowering LLMs to Understand and Generate Complex Vector Graphics. *arXiv preprint arXiv:2412.11102* (2024).
- [34] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu, and Qian Yu. 2025. Empowering llms to understand and generate complex vector graphics. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19487–19497.
- [35] Ximing Xing, Juncheng Hu, Jing Zhang, Dong Xu, and Qian Yu. 2024. Svgfusion: Scalable text-to-svg generation via vector space diffusion. *arXiv preprint arXiv:2412.10437* (2024).
- [36] Ximing Xing, Chuang Wang, Haitao Zhou, Jing Zhang, Qian Yu, and Dong Xu. 2023. Diffsketcher: Text guided vector sketch synthesis through latent diffusion models. *Advances in Neural Information Processing Systems* 36 (2023), 15869–15889.
- [37] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. 2024. Svgdreamer: Text guided svg generation with diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4546–4555.
- [38] Yiming Xu, Qi Song, Yihan Wang, Wangqiu Zhou, and Junli Liang. 2025. LGC-CR: Few-shot Knowledge Graph Completion via Local Global Contrastive Learning and LLM-Guided Refinement. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 3688–3697.
- [39] Yiying Yang, Wei Cheng, Sijin Chen, Xianfang Zeng, Fukun Yin, Jiaxu Zhang, Liao Wang, Gang Yu, Xingjun Ma, and Yu-Gang Jiang. 2025. Omnsvg: A unified scalable vector graphics generation model. *arXiv preprint arXiv:2504.06263* (2025).
- [40] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chengxing Xie, Cunxiang Wang, et al. 2026. GLM-5: from Vibe Coding to Agentic Engineering. *arXiv preprint arXiv:2602.15763* (2026).
- [41] Peiying Zhang, Nanxuan Zhao, and Jing Liao. 2024. Text-to-vector generation with neural path representation. *ACM Transactions on Graphics (TOG)* 43, 4 (2024), 1–13.
- [42] Xuanle Zhao, Deyang Jiang, Zhixiong Zeng, Lei Chen, Haibo Qiu, Jing Huang, Yufeng Zhong, Liming Zheng, Yilin Cao, and Lin Ma. 2025. Vincicoder: Unifying multimodal code generation via coarse-to-fine visual reinforcement learning. *arXiv preprint arXiv:2511.00391* (2025).
- [43] Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, et al. 2025. Swift: a scalable lightweight infrastructure for fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 29733–29735.
- [44] Dawei Zhu, Rui Meng, Yale Song, Xiyu Wei, Sujian Li, Tomas Pfister, and Jinsung Yoon. 2026. PaperBanana: Automating Academic Illustration for AI Scientists. *arXiv preprint arXiv:2601.23265* (2026).
- [45] Minjun Zhu, Zhen Lin, Yixuan Wang, Panzhong Lu, Qiujie Xie, Yifan Wei, Sifan Liu, Qiyao Sun, and Yue Zhang. 2026. AutoFigure: Generating and Refining Publication-Ready Scientific Illustrations. *arXiv preprint arXiv:2602.03828* (2026).

Appendix

A Overview

In this supplementary material, we provide additional details and evaluations, including:

- Data Collection Pipeline.
- Detailed Prompt Templates.
- Human Evaluation.
- Additional Comparisons and Results.
- Evaluation Protocols.
- Rationale for Adopting SVG as the Target Representation.
- More Qualitative Results.

B Data Collection Pipeline

To bridge the complexity, logic, and structural gaps in existing SVG datasets for professional scenarios, we introduce a large-scale, structured SVG dataset specifically designed for meeting minutes and office documents. This dataset comprises 100,000 high-quality "office text-SVG diagram" pairs, comprehensively covering diverse Text-to-SVG tasks derived from real-world meeting minutes, natural dialogues, and professional documents.

Data Collection. To ensure authenticity and diversity, text captions are extracted from real-world sources, including meeting records, professional office documents, and the Arena conversation dataset. This multi-source strategy guarantees that the instructions reflect actual requirements in professional collaboration and documentation contexts.

Data Inference and Quality Grading. To maintain high-fidelity SVG code, we implement an automated yet quality-controlled data construction pipeline. Since different types of diagrams have different structural requirements, we adopt type-specific construction pipelines for different data categories, rather than relying on a single unified prompt template. For each caption, Gemini-3-Pro first generates two independent SVG candidates. A Judge Model then evaluates each candidate through multi-dimensional criteria, including visual correctness, layout clarity, topology validity, and semantic consistency with the input caption.

Based on the scoring results, we stratify the data into three difficulty levels:

- **Simple:** Both samples achieve perfect scores, indicating high generation consistency and relatively stable layout requirements.
- **Medium:** Only one sample reaches the perfect standard, while the other contains minor visual or structural flaws.
- **Hard:** Both samples exhibit flaws, usually corresponding to inputs that require more complex logical reasoning, dense information organization, or sophisticated layout control.

Hard Instance Repair. Identified hard instances are further processed by a specialized Repair Model. Given the imperfect SVG and the feedback from the Judge Model, the Repair Model iteratively revises the SVG code until it satisfies the predefined perfect-output standard. Only samples that pass the final quality check are retained in the dataset.

Bias Reduction and Human Verification. To reduce construction bias and avoid redundant samples, we conduct deduplication after data generation and repair. In addition, we perform stratified

human verification across difficulty levels: 30% of the samples in each difficulty group are manually audited to check visual quality, semantic faithfulness, and layout correctness. This ensures that the dataset quality is not solely determined by automatic model judgments. For evaluation, the 320 test cases are collected from real documents and meeting records outside the training set, and all test samples are fully manually validated before being used for benchmarking.

Data Distribution. The final dataset comprises 100,000 high-standard samples. The data spans six primary categories: *Explanatory Vis*, *Document Authoring*, *Meeting Summary*, *Data Vis*, *Others*, and *Icons*. Notably, *Explanatory Vis* constitutes the largest portion with 59,137 samples. Its granular sub-categories include Flowcharts (17.3%) and Architectural Diagrams (11.6%), which meet the demand for generating complex, document-level logical illustrations. Experiments demonstrate that this dataset effectively enhances model performance in visual semantic understanding and SVG code generation tasks, providing a solid foundation for related research.

C Detailed Prompt Templates

Prompt: SVG Generator Agent

You are an AI assistant specialized in generating SVG illustrations.

Requirements:

- Generate a professional SVG based on the provided user text, scenario (e.g., meeting review or office report), and diagram type (e.g., Flowchart, Mind Map, or Meeting Cover figure).
- The style must be professional, clean, and high-contrast.

Constraint Knowledge Checklist (must avoid):

- Element overlap (nodes, text, or edges intersecting or occluding each other).
- Incorrect connections (wrong arrow direction, broken/misaligned links, illogical connections, overly complex paths).
- Text overflow (text exceeding container boundaries or improperly wrapped).
- Cropped content (improper viewBox or insufficient canvas margins causing clipping).
- Severe alignment or style inconsistency (misalignment, uneven spacing, inconsistent styling).
- Factual inconsistency (fabricating ungrounded information, omitting key entities, or logical contradictions with the source text).

Ensure the generated SVG code satisfies all constraints and contains none of the above issues.

Output Format:

- The output must be a complete SVG code block.
- The code must be concise, well-structured, and visually clean.
- The SVG must be written in valid XML format.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 X Y">
...
</svg>
```

Figure 10: Prompt used for structured SVG generation.

The SVG Generator prompt defines the core generation policy of our system, as shown in Figure 10. It specifies structural constraints,

Prompt: SVG Verify Agent

Role. You are an experienced design consultant specializing in SVG standards, frontend visualization engineering, and advanced visual aesthetics.

Objective.

- Evaluate the rendered SVG result based on the caption, SVG code, and rendered image.
- Identify severe issues and attribute them to predefined categories.
- Determine whether the SVG requires repair.
- Provide concise and actionable improvement suggestions.

Predefined Issue Categories.

- Element overlap
- Severe alignment/style issues
- Incorrect connections
- Unreasonable layout (imbalanced or chaotic structure)
- Text overflow
- Cropped or incomplete content

Evaluation Principles.

- Focus only on severe issues that affect readability or visual integrity.
- Minor pixel-level misalignment or stylistic variations do not require repair.

Output Format. Output must strictly follow the structured XML format:

If repair is required:

```
<attribute>
[Issue Name]
</attribute>
<problems>
[Concise analysis and targeted suggestions]
</problems>
<is_repaired>Yes</is_repaired>
```

If no repair is required:

```
<is_repaired>No</is_repaired>
```

Figure 11: Prompt used for SVG aesthetic and standardization evaluation.

visual style requirements, and a constraint knowledge checklist to prevent common layout and rendering errors. By explicitly encoding design rules and output formatting requirements, this prompt guides the model to produce clean, well-structured, and fully renderable SVG code under standardized generation constraints.

The SVG Verify prompt functions as a structured evaluation agent (Figure 11). It analyzes the rendered SVG result with respect to predefined issue categories, including overlap, misalignment, layout disorder, and structural inconsistencies. By focusing exclusively on severe issues that compromise readability or visual integrity, the verify agent ensures objective feedback for downstream refinement.

As shown in Figure 12, the Factual Consistency prompt evaluates the semantic alignment between the input caption, and the rendered SVG image across three dimensions: Zero Hallucination

Prompt: Factual Consistency Evaluator

Role. You are a professional content audit expert specializing in evaluating the factual consistency between generated diagrams and their source text (Caption).

Objective. Based on the provided Caption (Ground Truth) and the rendered image, determine if the image is factually accurate across three dimensions. Assign a binary factual score: **0 or 1**.

Evaluation Dimensions. The image must **simultaneously satisfy** all three dimensions to receive a score of 1:

- **1. Zero Hallucination (Strict Anchoring):** Every text node must have a direct source in the caption. No inferred intermediate steps or principles. *Whitelist:* English translations, Subgraph containers, "End" nodes, and accurate summary titles.
- **2. No Omission:** Core stakeholders, key steps, and explicit end states must be present. (Exception: Data from logically different dimensions/scales are not considered missing).
- **3. Logical Consistency:** Hierarchical branching must use consistent classification dimensions. The diagram type (Flowchart, Sequence, etc.) and connection relationships must strictly align with the caption.

Scoring Criteria.

- **Score 1:** The image faithfully meets all three dimensions without any hallucination, omission, or logical error.
- **Score 0:** The image contains any hallucination (outside the whitelist), missing core content, or hierarchical/relational inconsistencies.

The score must be strictly binary (0 or 1).

Output Format. First provide a detailed analysis of the three dimensions within `<reason></reason>`, then provide the final score within `<score></score>`.

```
<reason>
- Hallucination Analysis: ...
- Omission Analysis: ...
- Logic & Hierarchy Analysis: ...
</reason>
<score>0 or 1</score>
```

No additional commentary is allowed.

Figure 12: Prompt for factual consistency evaluation.

(preventing unmentioned information), No Omission (ensuring core elements are present), and Logical Consistency (verifying hierarchical and structural accuracy). It performs element-level matching and assigns a binary factual score, ensuring the generated SVGs remain strictly faithful to the intended description.

The SVG Repair prompt implements a targeted correction mechanism (Figure 13). Guided by the Verify Agent's feedback, it converts abstract issue descriptions into concrete code-level modifications, recalculates layout when necessary, and eliminates structural and visual defects. The repair agent strictly outputs corrected SVG code, forming a closed-loop refinement pipeline.

Prompt: SVG Repair Agent

Role. You are a rigorous SVG repair engineer and code optimization expert.

Objective. Given the original caption, the initial svg code, the Verify Agent's attribute tags, and the detailed problems analysis, strictly follow the Verify Agent's feedback and precisely fix all identified issues while preserving correct semantics and structure.

Core Tasks.

- Convert each reported issue into concrete code-level modifications (e.g., adjusting coordinates, sizes, paths, alignment, text layout).
- Recalculate layout for alignment or spacing problems.
- Ensure responsive design by optimizing the viewBox and removing fixed width/height constraints.
- Eliminate overlap, overflow, clipping, and connection errors completely.

Quality Requirements.

- Maintain semantic accuracy with respect to the caption.
- Ensure visual clarity, modern flat style, and aesthetic consistency.
- Produce clean, well-structured, fully renderable SVG code.

Output Constraint. Return only the repaired SVG wrapped in:

```
<repaired_svg>
<svg ...>
...
</svg>
</repaired_svg>
```

No additional explanation or text is allowed.

Figure 13: Prompt used for SVG repair and refinement.

D Human Evaluation

As VLM-based evaluation cannot fully capture the subtleties of human aesthetics, we invited 10 UI designers to perform assessment. To ensure objectivity, each image was independently rated by two evaluators. If significant scoring discrepancies occurred, a senior UI designer intervened to make the final determination. The assessment followed a strict double-blind testing procedure, with each evaluator independently completing professional judgments on the following two dimensions:

- **Semantic Match:** Measures the semantic alignment between the generated SVG and the input text across three dimensions: Zero Hallucination, No Omission, and Logical Consistency. A score of 1 is awarded only if the SVG satisfies all three criteria, whereas a score of 0 is assigned if it contains hallucinations, missing content, or structural errors. To ensure comparability, the final average score is normalized to a 10-point scale.
- **Visual Quality:** Evaluates structural and aesthetic integrity by identifying specific visual defects, including element overlap, connection issues, text overflow, color coordination, and content occlusion. A deductive scoring mechanism is employed based on the presence and severity of these errors, where any severe

defect precludes a high score. The final evaluation is mapped to a discrete scale of 10, 8, 6, 4, or 2 points.

E Additional Comparisons and Results

We provide additional quantitative results on the DocMeetSVG-100K dataset.

Analysis of Dual Rendering Rewards. To further validate the necessity of our dual-reward mechanism, we qualitatively and quantitatively compare the generation results under different reward configurations (Figure 14). While the initial model (a) suffers from visually unappealing text overflow and rudimentary layouts, optimizing solely with a visual reward (b) inadvertently encourages the model to exploit shortcuts. This phenomenon, which we term "over-simplification," is clearly reflected in the skewed distribution of SVG path-related tags (Figure 14 (b), top), where the model heavily favors overly simplistic structures to easily satisfy basic aesthetic constraints.

In contrast, the integration of dual rendering rewards acts as a rigorous structural constraint that effectively balances graphical complexity and aesthetic layout. By penalizing such shortcuts, the path tag distribution smoothly shifts toward higher complexity (Figure 14 (c), top), guiding the model to generate structurally rich diagrams. This stable optimization process is further corroborated by our training metrics. As shown in Figures 15(a) and 15(b), both visual and complexity rewards exhibit steady growth, surpassing 0.82 and 0.90 respectively. Notably, a larger rollout size (Rollout=32) yields more stable convergence with a faster reduction in variance. Crucially, Figure 15(c) demonstrates that completion lengths remain stable (3,000–4,000 tokens) throughout this process, firmly indicating that the model optimizes spatial layouts without resorting to degenerate shortcuts to inflate visual scores. Finally, the robustness of our approach is evident in the accept ratio, where GVR-Coder improves steadily to 0.67 (Figure 15(d)), outperforming the DPO baseline (0.24). This gap stems from DPO's offline nature, which evaluates sequences holistically and fails to assign precise credit to the specific local tokens responsible for visual errors.

F Evaluation Protocols

Aesthetic and Normative Scoring: Standard Text-to-Image metrics such as FID and CLIPScore often correlate poorly with the logical and topological requirements of document-level diagrams. We therefore adopt Gemini-3-Pro as VLM-as-Judge to provide nuanced visual feedback.

To bridge the gap between automated checking and manual review, we classified errors into three dimensions and six types (Figure 16). We utilize a deductive scoring mechanism: $Score = \max(0, 6 - N)$ where N represents the number of detected errors.

Factuality Scoring: The score is assigned to measure semantic alignment across three core dimensions: Zero Hallucination, No Omission, and Logical Consistency. 1 Point: The SVG satisfies all three criteria, faithfully reflecting the caption without redundant content. 0 Point: The image fails in any dimension. For more detailed prompts, please refer to the supplementary materials.

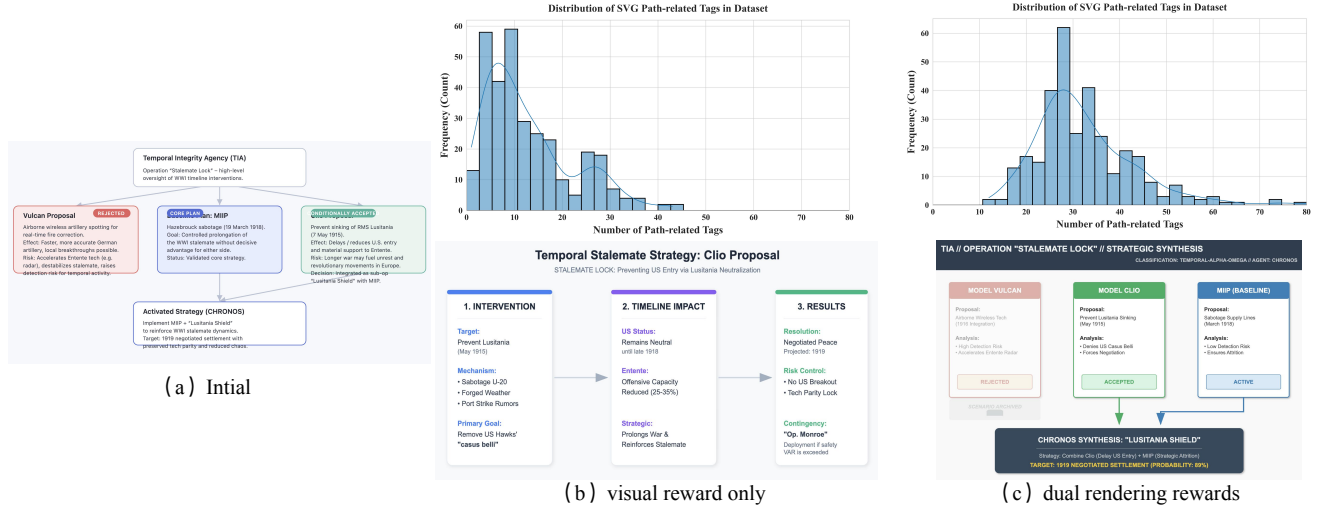


Figure 14: Effectiveness of dual rendering rewards.

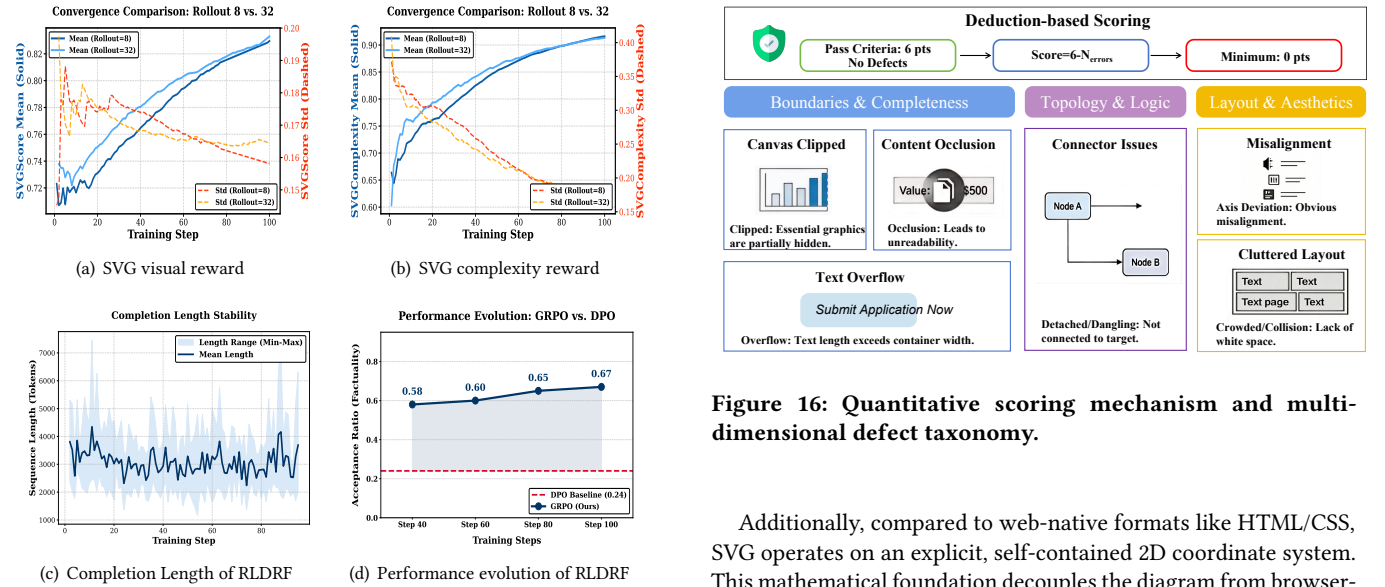


Figure 15: Analysis of RLDRF Training Dynamics on n320 datasets.

G Rationale for Adopting SVG as the Target Representation

We adopt Scalable Vector Graphics as our primary diagram representation due to its unique balance of visual expressiveness, precise spatial control, and widespread cross-platform compatibility.

While declarative graph languages (e.g., Mermaid) are efficient for standard charts, they are heavily constrained by predefined routing algorithms and fixed templates. In contrast, SVG natively supports free-form layouts, nested architectural cards, customized iconography, and complex visual styling such as gradients, enabling fine-grained design far beyond rigid typologies.

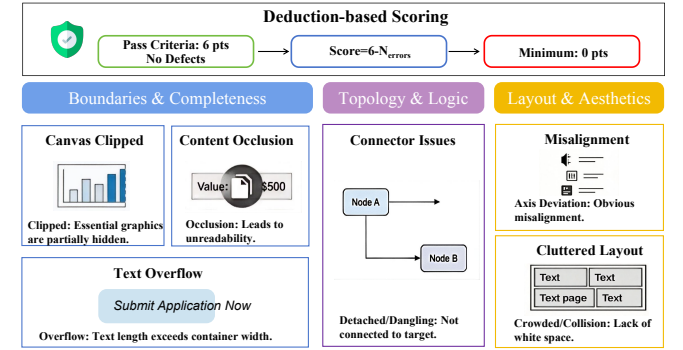


Figure 16: Quantitative scoring mechanism and multi-dimensional defect taxonomy.

Additionally, compared to web-native formats like HTML/CSS, SVG operates on an explicit, self-contained 2D coordinate system. This mathematical foundation decouples the diagram from browser-specific DOM rendering or external stylesheets. Consequently, SVG guarantees higher spatial freedom and native scalability, allowing the generated assets to be seamlessly inserted, resized, and manually refined in diverse downstream environments, including documents, presentation slides, and vector graphics editors. Ultimately, SVG serves as a practical medium that bridges the gap between programmatic logic generation and aesthetic visual design.

H More Qualitative Results

In this section, we provide additional qualitative visualizations to further demonstrate the effectiveness and robustness of our proposed framework. Figure 17, 18, and 19 present an extended visual comparison between our model and state-of-the-art closed-source LLMs (i.e., Gemini-3-pro) across diverse and complex diagram types. The results clearly highlight our model's superiority

in maintaining strict layout alignment, hierarchical clarity, and aesthetic design. Furthermore, Figure 20 provides detailed, step-by-step examples of the Generate-Verify-Repair agent loop in action. It illustrates how the verifier model accurately identifies specific

visual defects—such as missing background elements or overlapping connection lines—and effectively guides the repair model to produce highly polished, deployment-ready SVGs.

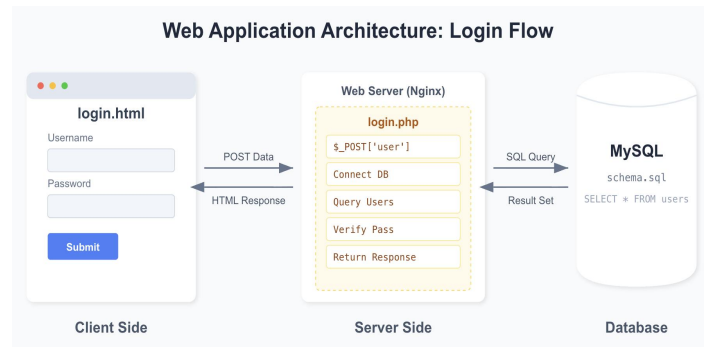
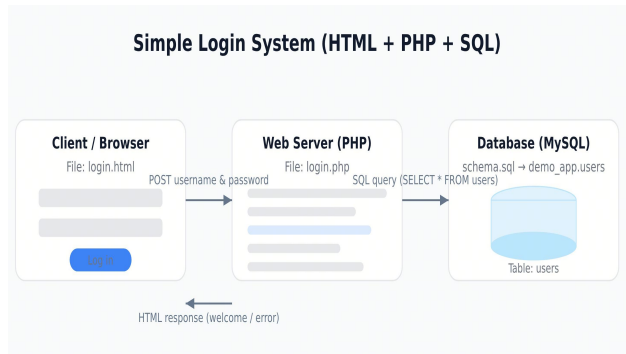


User: Create a simple login page using separate SQL and PHP files.

Assistant: We provide a minimal login system with three independent files for demonstration:

login.html: A basic login form that sends username and password to login.php. **schema.sql:** SQL script for creating the database, user table, and a test account.

login.php: PHP script for database connection and credential verification. Security note: This is an educational example; production systems require prepared statements, password hashing, and other protections.



At this meeting, the film review company provided film academy students with high-quality learning materials by delivering professional analyses of the educational significance, performing arts, and profound themes of several classic films. The discussion covered "A Chinese Odyssey," a quintessential representative of Stephen Chow's artistry that uses a comedic shell to tell a poignant tragedy of star-crossed lovers separated by time and space; "Wolf Warrior 2," which showcases the responsibility of Chinese soldiers and national pride through a justice-driven narrative; and "Looking Up," a reflective piece on education advocating that there are no useless children, only improper teaching methods. The core consensus reached is that outstanding cinematic works are not merely entertainment but vital artistic vehicles for patriotism, philosophy, and social reflection, and should be promoted as essential teaching resources for in-depth academic study.

Film Analysis & Educational Value Exploration

Key Work Deep-Dive

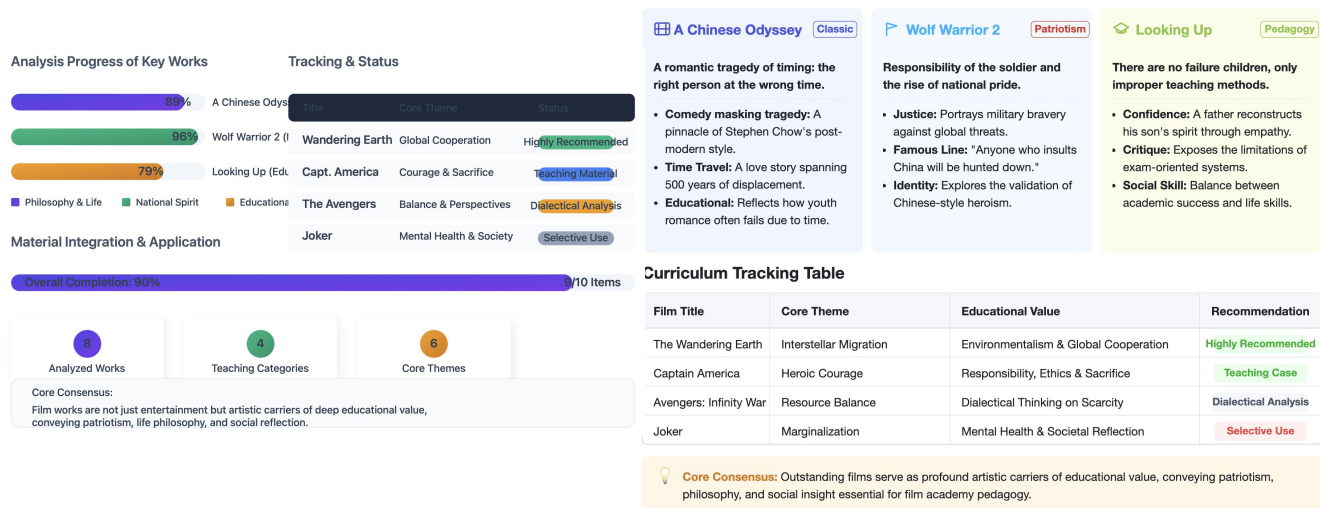
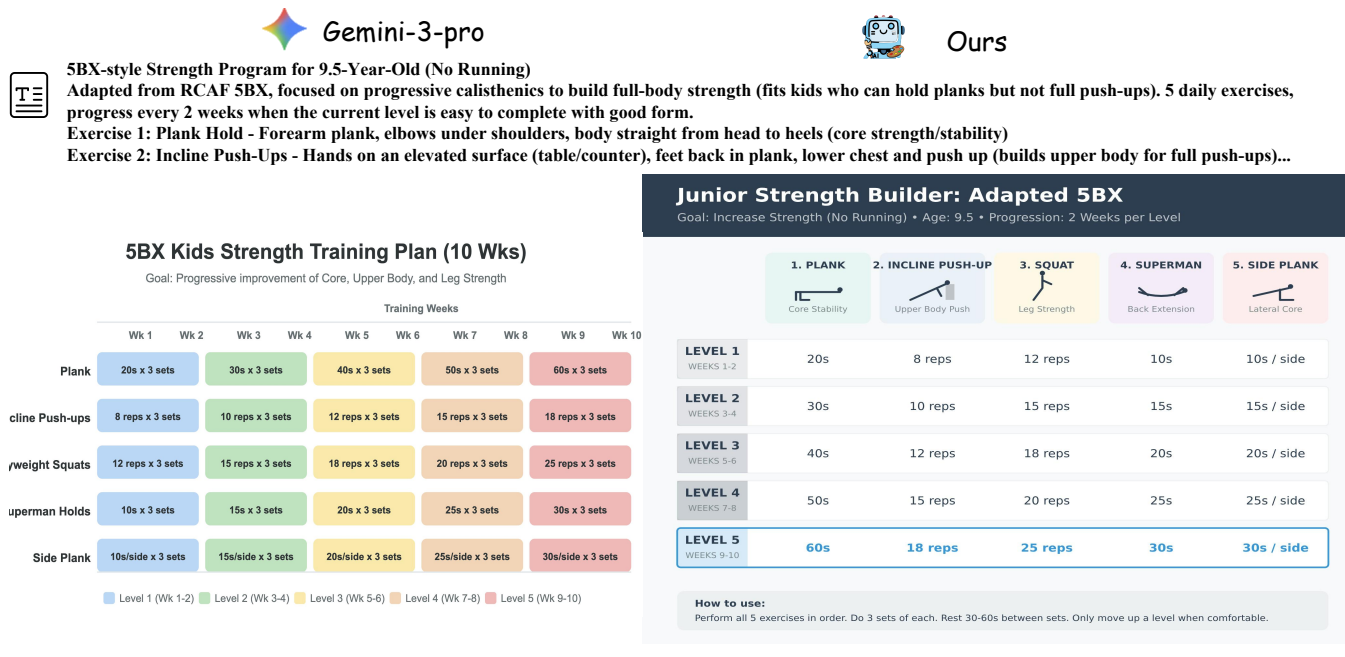


Figure 17: Comparative visualization results between Gemini-3-pro and GVR-Coder



The working mechanism of omnichannel Q&A is a closed-loop iterative process: AI Q&A and customer service Q&A are first deployed upfront to interact with users. If the AI cannot resolve the inquiry, the issue is escalated to human customer service. Valuable knowledge generated from human customer service interactions is then accumulated into the knowledge base. In subsequent AI Q&A sessions, the newly added content in the knowledge base can be invoked to improve answer accuracy, forming a continuously optimized cycle.

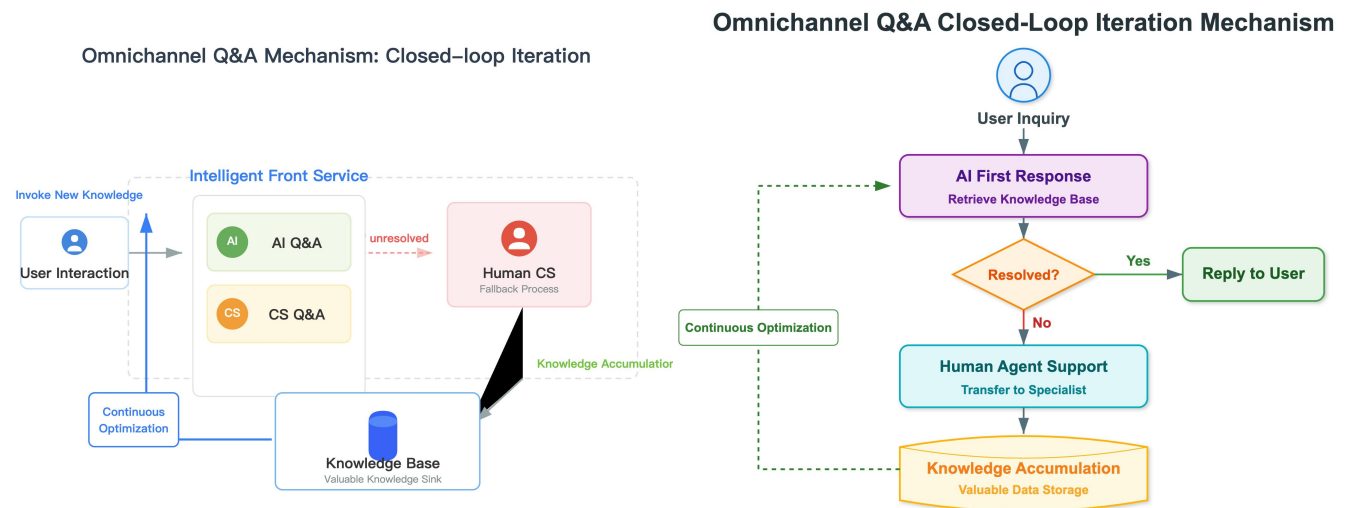
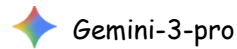


Figure 18: Comparative visualization results between Gemini-3-pro and GVR-Coder



Ours



There are three adjustments to the Q4 policy for existing customer advocacy: 1. Incentives now match event levels, where P0 rewards are doubled, P1 remains unchanged, and P2 offers no incentive (levels determined by Marketing); 2. Iteration of dinner scenario incentives, where vouchers can be requested for existing customers if the total attendance is ≥ 6 and the ratio of new to existing customers is at least 4:1; 3. The customer speaker list has shifted from quarterly updates to dynamic updates, requiring joint confirmation from A and B and group chat approval to be added to the whitelist for voucher eligibility. This policy is effective starting Q4, with next year's adjustments TBD.

Q4 Existing Customer Referral Policy Adjustments		
Core Changes		Key Criteria / Process
1	Incentive Standards Linked to Event Level P0 Level: Double Rewards P1 Level: No Change P2 Level: No Incentive	Level determined by Marketing Effective from Q4
2	Dinner Scenario Iteration Vouchers for Existing Customers	Must meet both criteria: · Total attendees ≥ 6 · New/Old ratio ≥ 4:1
3	Speaker List Update Quarterly → Dynamic Updates Eligibility after Whitelisting	1. Joint confirmation (A & B) 2. Group chat approval 3. Added to Whitelist

Q4 Existing Customer Advocacy Policy Adjustments

1. Incentive Matching

- P0 Event** Double Reward
- P1 Event** No Change
- P2 Event** No Incentive

Note: Event level is determined by Marketing team members.

2. Dinner Scenario Iteration

Application Criteria:

- Total Customers ≥ 6
- New:Old Ratio $\geq 4:1$

↓

Apply for Voucher

3. Dynamic List Updates

Change Description:

Was: Quarterly window

Now: Dynamic updates

Approval Process:

Joint Confirmation (A & B)

↓

Group Chat Approval

↓

Add to Whitelist

17 Execution: Starts Q4 | Next year adjustments TBD



The evaluation framework for legal personnel and lawyers is structured around a multi-dimensional assessment approach. For legal staff, the system is bifurcated into hard metrics, which track performance-driven data such as client visit volumes and contract execution counts, and soft metrics, which encompass qualitative factors including client service evaluations, synergy with lawyers during case proceedings, and contributions to organizational knowledge sharing. Correspondingly, the evaluation for lawyers integrates operational efficiency indicators—such as the lead time from case acceptance to filing and mediation success rates—with satisfaction ratings from both external clients and internal legal personnel, alongside intellectual contributions through landmark case studies. To enhance regulatory precision, the appraisal cycle is being optimized from a traditional annual review to a quarterly or semi-annual frequency, leveraging systematic data collection to mitigate human statistical errors and ensure objective oversight.

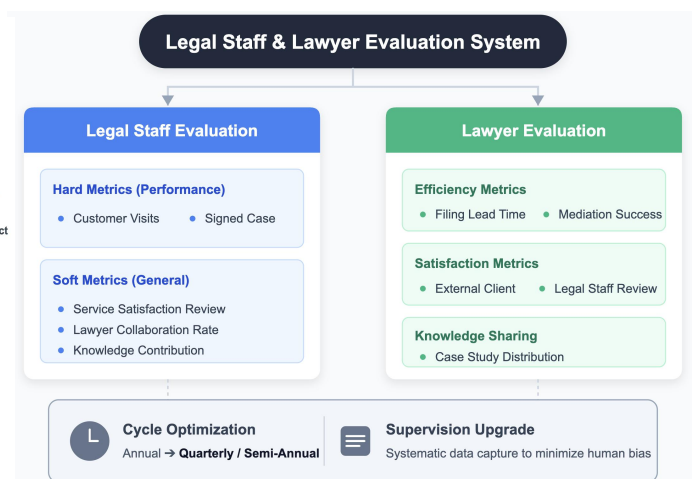
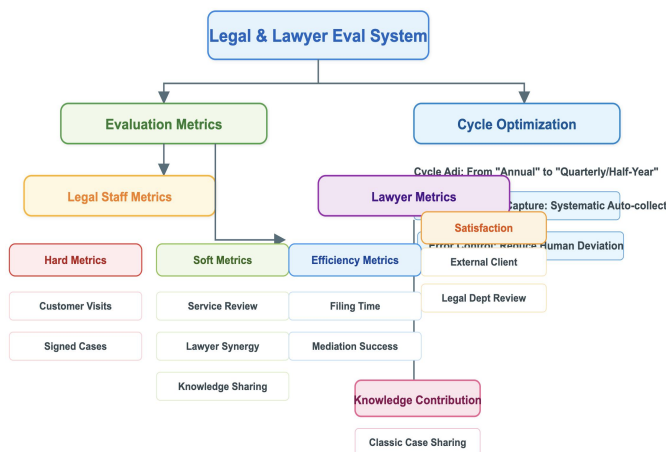
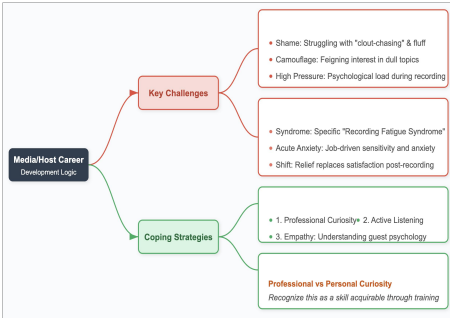


Figure 19: Comparative visualization results between Gemini-3-pro and GVR-Coder

Caption

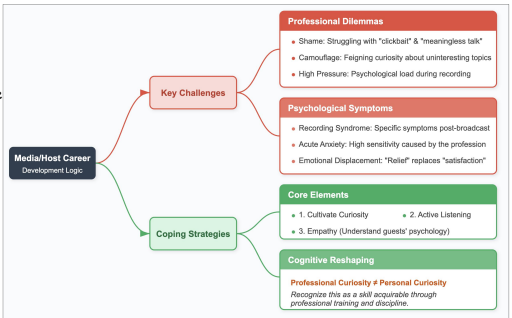
Media professionals and hosts face two major categories of challenges in their career development. In terms of career dilemmas, these include the sense of shame associated with "jumping on bandwagons and talking nonsense".....

Generate

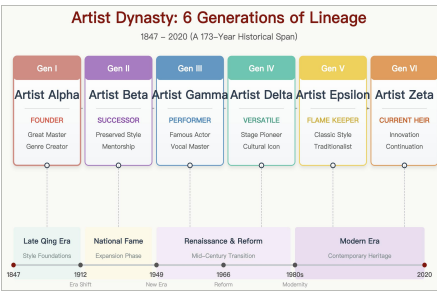


attributes: ["Alignment & Style Issue"],
problems: The title background areas of the four detail nodes on the right have completely disappeared, causing the white title text to blend into the white card background and become completely invisible. This issue is caused by an incorrect setting of the clip-path parameter for the title background rectangles in the code—the bottom clipping value of 104px exceeds the rectangle height of 36px.
score: 5

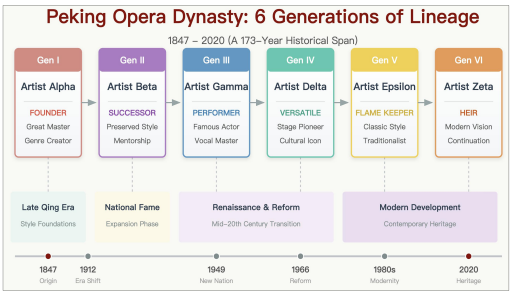
Repair



The six-generation inheritance pedigree of the Tan family's Peking Opera heritage presents a complete lineage spanning from the first to the sixth generation. The first-generation founded the Tan School:



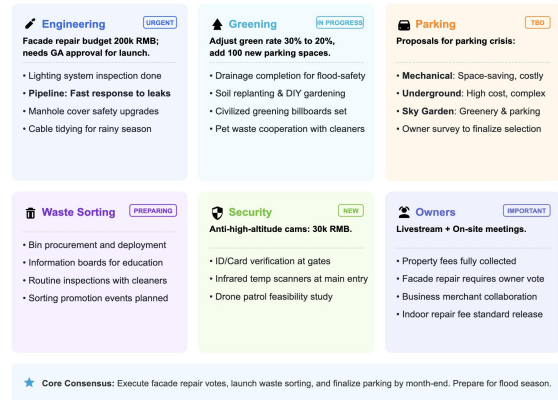
attributes: ["Element Overlap", "Connection Issues"],
problems: Connection Line Occlusion and Path Errors: The intergenerational connection lines are placed underneath the card layers and completely obscured by the card backgrounds, making the inheritance relationships invisible. Moreover, the connection line paths are designed to run through the center of the cards, which would impair text readability if displayed. Insufficient Layout Space: The spacing between cards is excessively narrow (only 10px), leaving no room for valid connecting arrows;
score: 4



Discussion on Community Facility Maintenance

Comprehensive planning for next month: Engineering, Greening, Parking, and Waste Sorting.

Key Discussion Topics



attributes: [null],
problems: null,
score: 6

Figure 20: Visualization of the Generate-Verify-Repair Agent Loop (VRL) process